

LAB 2: Requirements Analysis

1. OBJECTIVES

The following objectives are to be started during Lab #2 and completed by the beginning of Lab #3 unless otherwise stated.

- 1.1 Refine Use Case model
- 1.2 Build the Conceptual Model
- 1.3 Build a Dynamic Model of the system
- 1.4 Technology stack discussion

2. INTRODUCTION

- 2.1 Starting from the Use Case descriptions, more details about the system requirements can be uncovered through requirement analysis. By building a model of the application (problem) domain, the team can validate, correct and clarify the requirements.
- 2.2 Requirements elicitation and analysis are highly iterative activities. Expect to re-visit your Use Case diagram and User Case descriptions to clarify and add detail, as you work through requirements analysis.
- 2.3 Using the methodology described in Bruegge Chapter 5, the team will develop the Analysis Model consisting of (a) the Object Model, also known as the Conceptual Model, and (b) the Dynamic Model.
- 2.4 The conceptual model is usually depicted in a class diagram, while the dynamic model is usually depicted as a state machine.
- 2.5 The analysis model forms the basis for system design and is to be refined in Lab 3.

3. PROCEDURE

3.1 *Refine Use Case Model*

- 3.1.1 Finalize use case diagram with clear definition of actors and use cases
- 3.1.2 Keep refining user case descriptions. Describe clearly use case precondition, flow of events, and alternative flows.
- 3.1.3 Expect to re-visit your Use Case diagram and User Case descriptions to clarify and add details as you build the conceptual model and dynamic model

3.2 *Build the Conceptual Model*

- 3.2.1 From the Use Case Descriptions and the terms in the Data Dictionary, identify the initial entity, boundary and control objects and their attributes.
- 3.2.2 From your understanding of the classes' responsibilities, apply the stereotypes for entity, boundary or control classes.
- 3.2.3 In the Class Diagrams, depict these classes and the relationships between them (association, generalization, aggregation).

3.3 *Build a Dynamic Model*

- 3.3.1 Based on the UI mockups developed in Lab#1, model the system behavior as a State Machine of UIs. This state machine of UIs is referred to as a *Dialog Map* of the system. If this process reveals deficiencies in your UI flow, refine and further specify your user interface design. See page 420 of Fox for a detailed description of Dialog Maps.
- 3.3.2 Expect to move back and forth between the Conceptual Model and the Dynamic Model because studying interactions and behavior will reveal additional attributes, operations and relationships of classes.

3.4 *Technology Stack Discussion*

- 3.4.1 Discuss among your group which programming language, libraries, and services you might use in implementing your system. Note that concrete commitments to particular technologies are not required at this stage. In software development, the list of selected technologies for development and running of the system are often referred to as the *technology stack*.
- 3.4.2 Upload your system description (from lab 1) and your class diagram to an AI chat interface, along with the following (or similar) prompt:

- > Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team.

You can add extra information to the prompt, such as technologies you are already familiar with ("We have previous experience with [Java... etc]"), and technologies you are particularly interested in learning ("We hope to gain experience with [the React framework... etc]").

Note You are free to use your preferred AI model to complete the above exercise, although it must be able to successfully read the uploaded files. At the time of

writing, the free tier of ChatGPT is capable of performing the exercise, although some usage limits exist on user messages per day. If you include the documents in a project open in your IDE, an IDE AI plugin such as Clide may also be able to read the files. If you find yourselves unable to complete this exercise due to restrictions on free use or file reading capabilities, please contact the course coordinator.

- 3.4.3 Do not simply obey the AI's recommendations! It is very likely that the AI will recommend a more complicated technology stack than is necessary for your project. The purpose of this task is to get you thinking about commonly used technologies in industry, NOT to fix the technology stack you will actually be using in future labs. If the AI recommends technologies you are unfamiliar with, you may find it interesting to get it to justify and explain its choices, or to read further online about how these technologies are typically used in industry.
- 3.4.4 Select one of the technologies proposed by the AI that is *not* a programming language, and write a short opinion (2-3 sentences) on whether it is appropriate for your project. You can take into account factors such as your team's existing expertise, and the limitations of a course project in contrast to a long-term commercial development.
- 3.4.5 Prepare a short PDF report containing (1) a brief description of the uploaded material [full reproductions not necessary], (2) your text prompt and the original AI's chat response, (3) your written opinion from 3.4.4.

4. DELIVERABLES

- Complete Use Case diagram
- Use Case descriptions
- Class diagram of entity classes
- Key boundary classes and control classes
- Initial Dialog map
- Your PDF report on the AI's technology stack recommendation.

Please submit the deliverables to your Github repository (under the folder "lab2") before the Lab#3 starts. Your Lab Supervisor will wish to see and discuss the deliverables with you during Lab #3.

Use Case descriptions, class diagram, and dialog map shall be finalized in the Lab#3.

5. REFERENCES

- Introduction to Software Engineering Design: Processes, Principles and Patterns with UML2 – C. Fox
- Object-Oriented Software Engineering: Using UML, Patterns and Java – B. Bruegge and A. Dutoit